

AI Review of the FNDR Project Design

Final AI-Generated Review Report

AI tool/model: ChatGPT — GPT-5.6 Sol

Overview. FNDR has a strong technical foundation as a local-first semantic memory system for macOS. The design presents a clear capture-to-retrieval pipeline using ScreenCaptureKit, Apple Vision OCR, local embeddings, LanceDB, and a Tauri interface. Its strongest architectural idea is the separation between transient visual capture and persistent semantic memory: screenshots are processed in memory, transformed into searchable representations, and discarded rather than retained.

The design document also shows that FNDR is evolving beyond semantic retrieval. The timeline introduces computer-use agents, VLM grounding, multi-step reasoning, task execution, human-in-the-loop safety gates, and future cloud integration. These capabilities are substantially different from the seven-module architecture formally described earlier in the document. The team's newer direction—using FNDR's memory as context for a constrained memory-to-action agent—makes this discrepancy more important.

Primary design risk

The core semantic-memory pipeline is well specified, but the document does not yet fully define the architecture, evaluation methodology, privacy boundary, and safety model required to turn retrieved memory into reliable actions.

1. Agentic functionality is not represented in the core architecture

The formal architecture defines seven modules centered on capture, OCR, embeddings, LanceDB, UI, privacy, and IPC. Yet the development timeline introduces VLM-based grounding, computer-use actions, multi-step reasoning, and autonomous task completion. This creates a major architectural gap. A memory-to-action system requires additional concepts that currently have no explicit place in the design:

- context selection and packaging;
- skill discovery and selection, including reusable skills/*.md instructions;
- orchestration across skills and tools;
- planner/model interaction and constrained action execution;
- task state, trajectory storage, verification, recovery, and evaluation results.

These should not be treated as incidental functionality inside the existing IPC or UI layers. FNDR needs a clear agent subsystem connecting semantic retrieval to action execution. A useful conceptual flow is: Capture → Semantic Memory → Retrieval → Context Assembly → Planner → Skill/Tool Orchestration → Action → Verification → Evaluation.

2. The privacy model conflicts with cloud-assisted reasoning

The document repeatedly states that FNDR performs all computation locally, makes no external ML API calls, requires no network connection for normal operation, and makes zero outbound requests. This is internally consistent for the semantic-memory system described in the main architecture and privacy sections.

The proposed agent direction changes this assumption. The team intends to use provided ChatGPT models as high-level planners while FNDR supplies selected retrieved context and performs local

execution. Even if screenshots and the complete memory database remain local, retrieved text, URLs, application names, timestamps, or other metadata may leave the device. The design therefore needs a more precise trust boundary.

The core local memory system can preserve its existing privacy guarantee, while experimental cloud-assisted agent functionality should be treated as a separate operating mode. Context sent to the planner should be minimized, filtered for sensitive categories, and clearly distinguished from data that always remains local. Without this distinction, the project's strongest privacy claim becomes ambiguous.

3. The testing strategy does not yet support evaluation-driven agent improvement

The current QA strategy is appropriate for the original system: retrieval correctness, OCR/embedding performance, memory behavior, and search latency. It is insufficient for the agentic system. The team's intended improvement loop is not primarily reinforcement learning in the conventional sense because there is currently no plan to update foundation-model weights. It is better characterized as evaluation-driven agent optimization: benchmark tasks expose failures, and those failures guide improvements to retrieval, prompts, skills, and orchestration.

- Retrieval — Was the necessary memory present and appropriately ranked?
- Planning — Did the model infer the correct objective and next step?
- Skill selection/orchestration — Did FNDR load the correct skills and tools?
- UI grounding/action selection — Did it choose the correct action and target?
- End-to-end completion — Did the overall task succeed?

Retrieval evaluation is especially important because FNDR's differentiation comes from its context layer. Failure cases should include missing relevant memories, low ranking, excessive irrelevant context, and situations where misleading retrieved context causes an incorrect action. The team should maintain a benchmark suite and compare results whenever prompts or skills change.

4. Agent permissions, confirmation, and failure recovery require a formal design

The timeline mentions safety stress testing and later introduces human-in-the-loop safety gates for destructive actions. That is a useful start, but the execution model remains under-specified. FNDR should not expose unrestricted general computer control during Alpha. The intended design is better represented as allowlisted workflows using explicitly available skills and tools.

For Alpha, requiring user confirmation before external actions provides a simple and defensible boundary while the system is being evaluated. The architecture can later support permissions at the skill or workflow level if reliability becomes sufficient. Skill definitions should include purpose, triggering conditions, available tools, workflow, constraints, success criteria, failure handling, examples, and verification requirements. Every revision should be versioned and tied to evaluation results so improvement is measurable.

5. The roadmap is broader than the defined product architecture

The timeline introduces several major systems that receive little or no treatment in the formal requirements: meeting recording, audio storage, GraphStore, knowledge-graph visualization, smart notifications, cloud/mobile bridges, local LLM optimization, and agent control. Individually, several are reasonable extensions. Collectively, they create substantial scope risk.

The strongest Alpha would prioritize the chain that validates FNDR's newer product hypothesis: stored context → useful retrieval → appropriate skill selection → successful constrained task. The team should still demonstrate at least one compelling multi-step end-to-end workflow, but the purpose

should be to test this architecture rather than establish broad general-purpose autonomy. Secondary systems should remain lower priority until this steel thread is reliable.

Additional Technical Concern

Several implementation choices should be validated experimentally rather than treated as fixed assumptions. In particular, applying PCA dimensionality reduction, Float16 storage, and Differential Privacy noise simultaneously could reduce retrieval quality. Because context relevance is now central to downstream agent success, the team should perform ablation evaluations comparing retrieval quality before and after each transformation. More broadly, architectural decisions in the agent stack should increasingly be justified by measured task performance rather than by expected efficiency or sophistication alone.

Final Recommendation

FNDR's most promising direction is not to replace its semantic-memory architecture but to build on it. The existing local capture and retrieval system can become the context layer for a constrained memory-to-action agent. Before Alpha development proceeds substantially further, the design should formalize four additions: an agent/orchestration architecture, a cloud/local privacy boundary, a versioned evaluation-and-improvement loop, and an explicit action-permission model.

The project should then reduce unrelated scope and evaluate one complete memory-to-action workflow in depth. This would make FNDR's architecture more internally consistent while preserving its strongest differentiator: an agent that can act using contextual knowledge derived from a user's own digital history rather than beginning each task without memory.